

CSOPESY -- 2 Processes

Concept

- OS executes variety of programs that run as process
- **Process** - a program in execution; process execution; process execution must progress in sequential fashion. No parallel execution.
 - If you do if-else execution in your code, what you're doing is jumping instructions. Never always split off na sabay-sabay
- Multiple parts
 - **Program Code**, also called a text section
 - instruction
 - Current activity including **program counter, process registers**
 - **Stack** containing temporary data
 - Function parameters, return address, local variables
 - **Data section** containing global variables
 - try to minimize global variables because of memory consumption
 - **Heap** containing memory dynamically allocated during run time
 - section that processes memory that is dynamically allocated
 - arrays or collections is an example

Conservation of memory is not that big of a concern for most developers because generally 8 GB is most common for everyone now. Developers are now as not as keen to improve memory management.

- Program is passive entity if stored on disk (.exe)
 - Program becomes process when executable file is loaded into memory
 - I.e., program started via double click
 - A program can be several processes - multiple users executing the same program

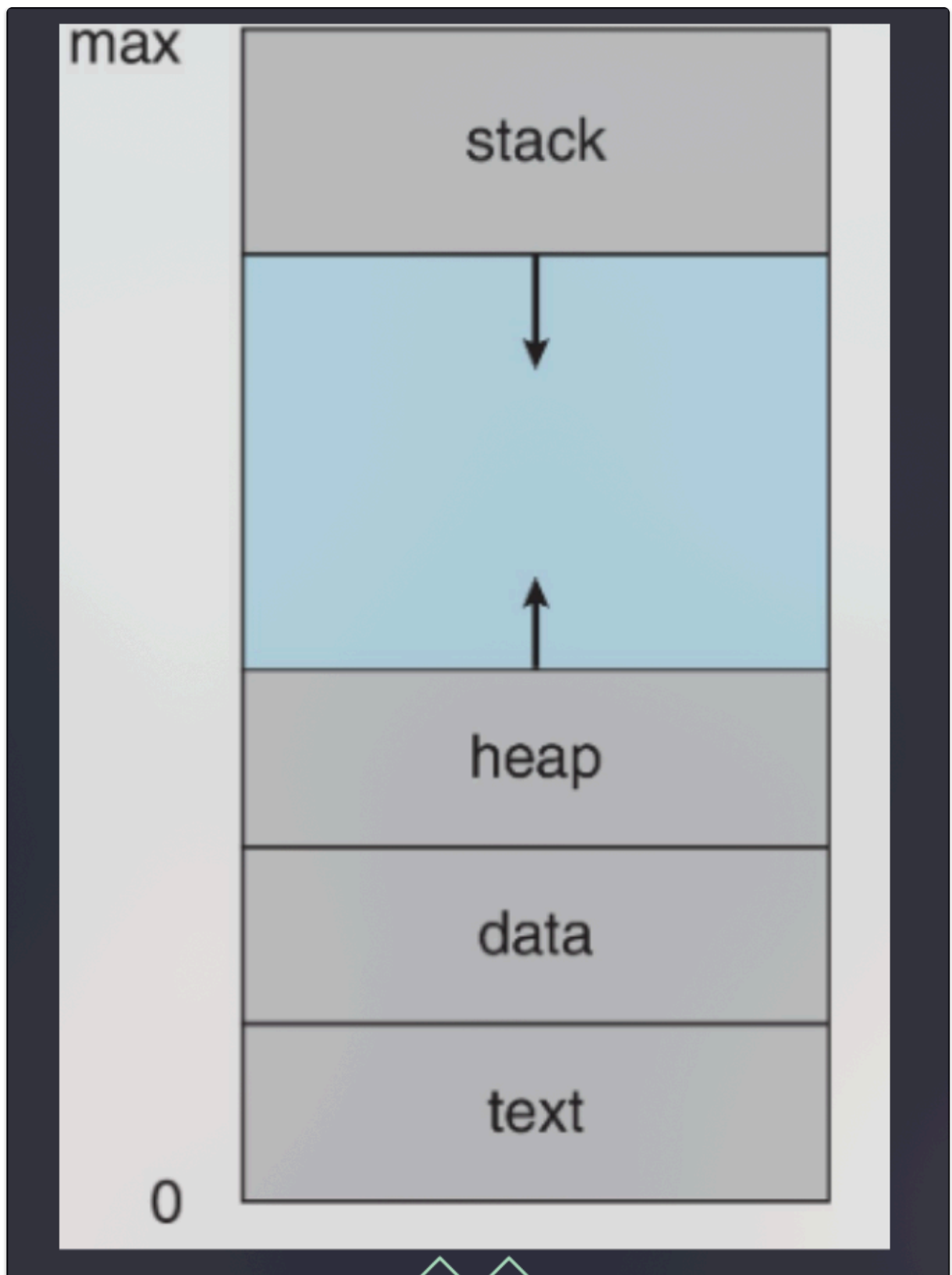


Image 1: Memory layout of most processes

Memory Layout of a C Program

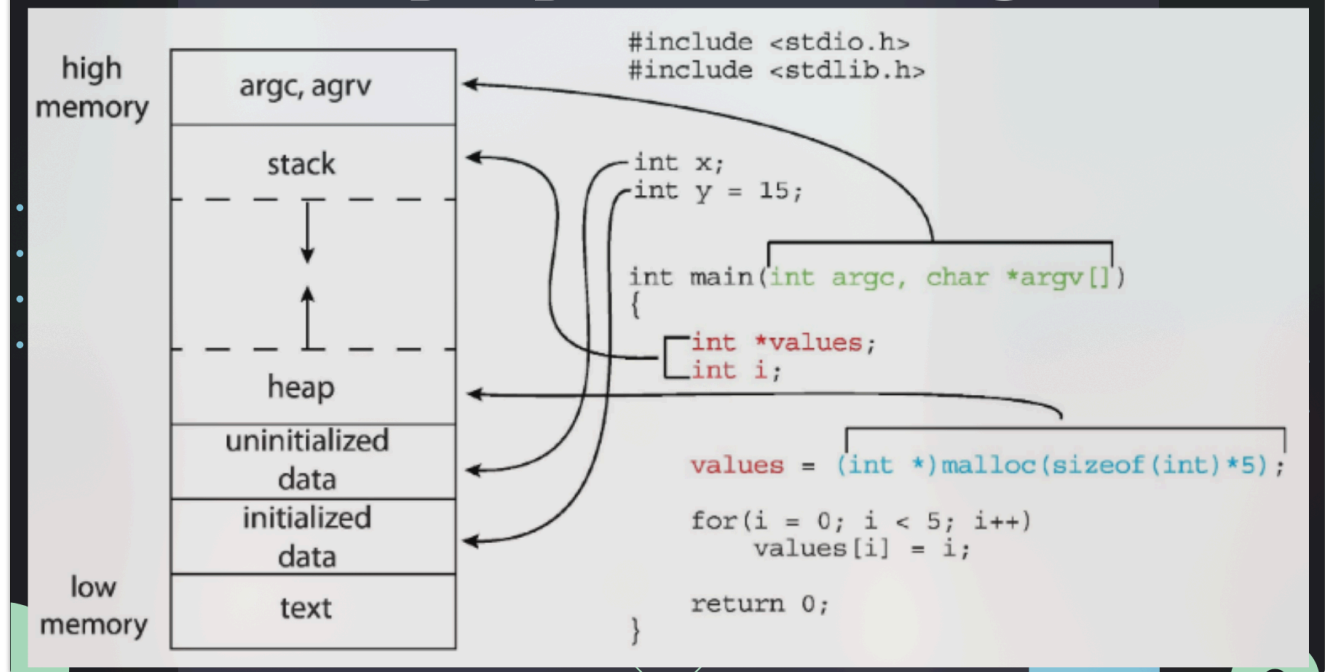


Image 2: Memory layout

- Text is always the predictable part of the memory (memory addresses)

Process State

Process changes state

1. New - process being created
2. Running - instructions executed
3. Waiting - process waiting for some event to occur
4. Ready - process waiting to be assigned to a processor
5. Terminated - process has finished

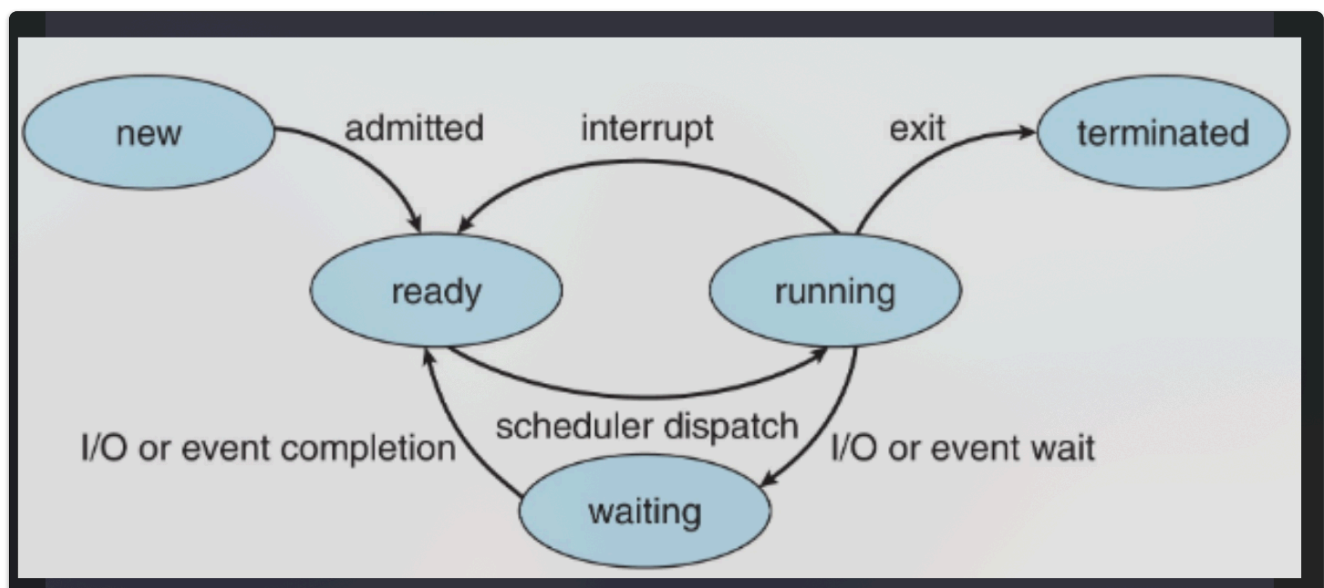


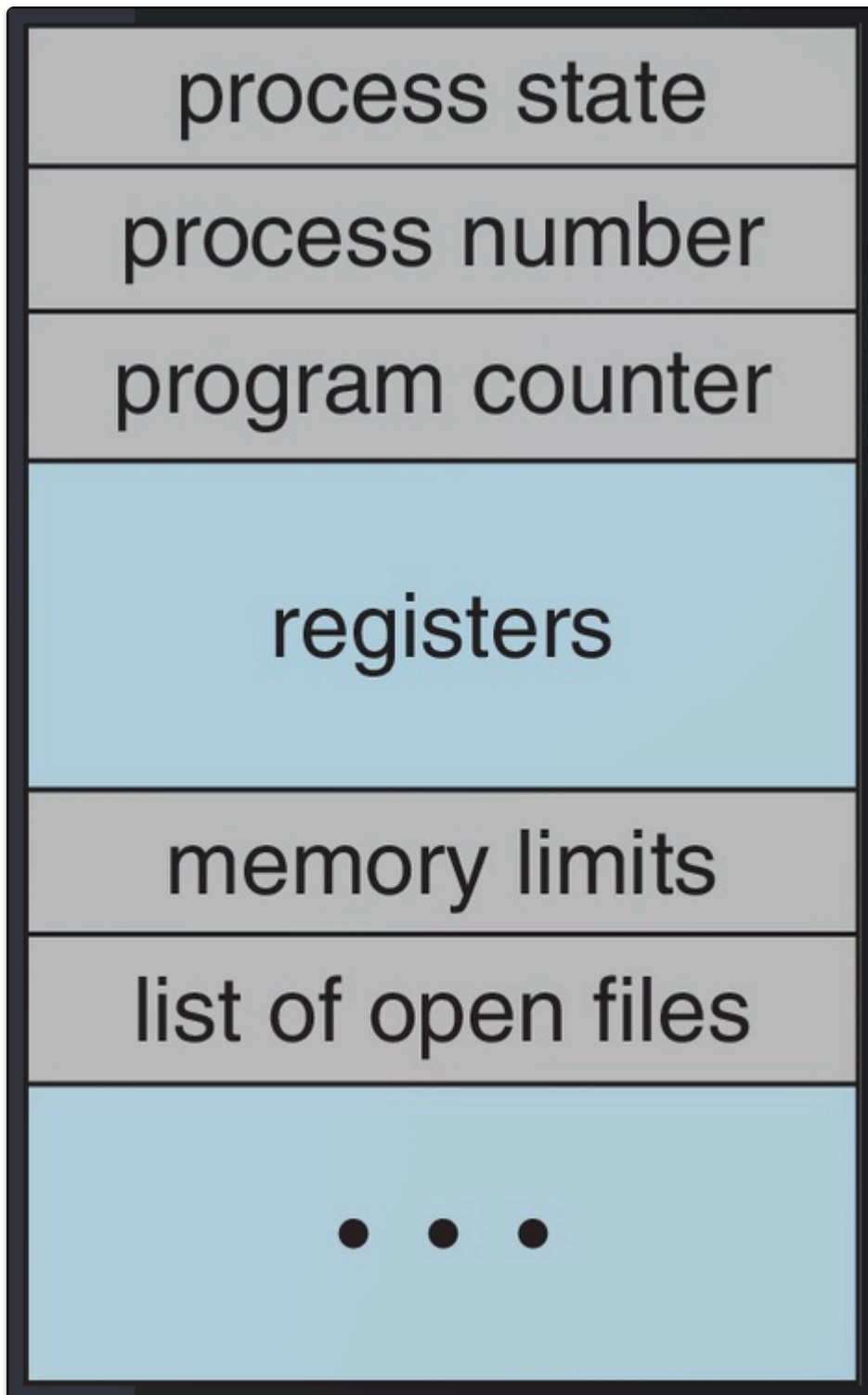
Image 3: Process State transitions

Ready Queue (the ready bubble) - waiting for the processes to let the cpu give them a turn.

There will come a point where you will stop because the CPU tells you that your time is up (interrupt transition)

Process Control Block

- Information associated with each process (**task control block**)
 - tracker of the operating system of the processes (index card)
 - Process state - running, waiting
 - Program Counter - location
 - CPU registers - keep tracks of registers
 - CPU scheduling information - priorities, scheduling queue pointers
 - Memory management information - memory allocated to process
 - Accounting information - CPU used, clock time, time limits
 - I/O Status information - I/O devices allocated to process



Threads

- Consider now that having multiple program counter per process
 - multiple locations can execute at once
 - process in a process
- multiple threads of control
- Tasks of execution that can happen simultaneously or concurrently
- Must need storage thread details multiple program counter in PCB

Process Scheduling

- The goal is to maximize CPU efficiency and use.
- Select among available processes in memory for next execution
- **Ready Queue** - set of all processes in main memory, ready for execution
- **Wait Queue** - set of processes waiting for event (i.e., I/O)
- Processes may migrate

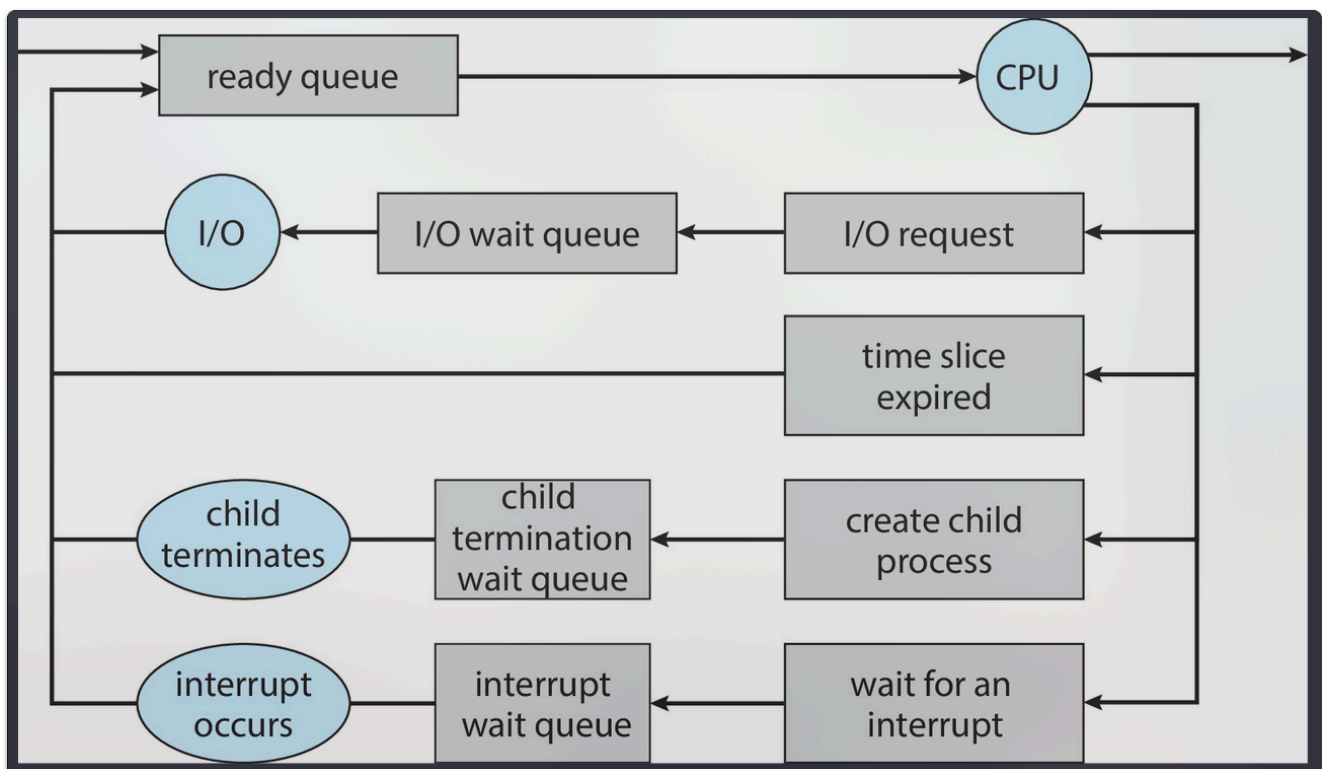
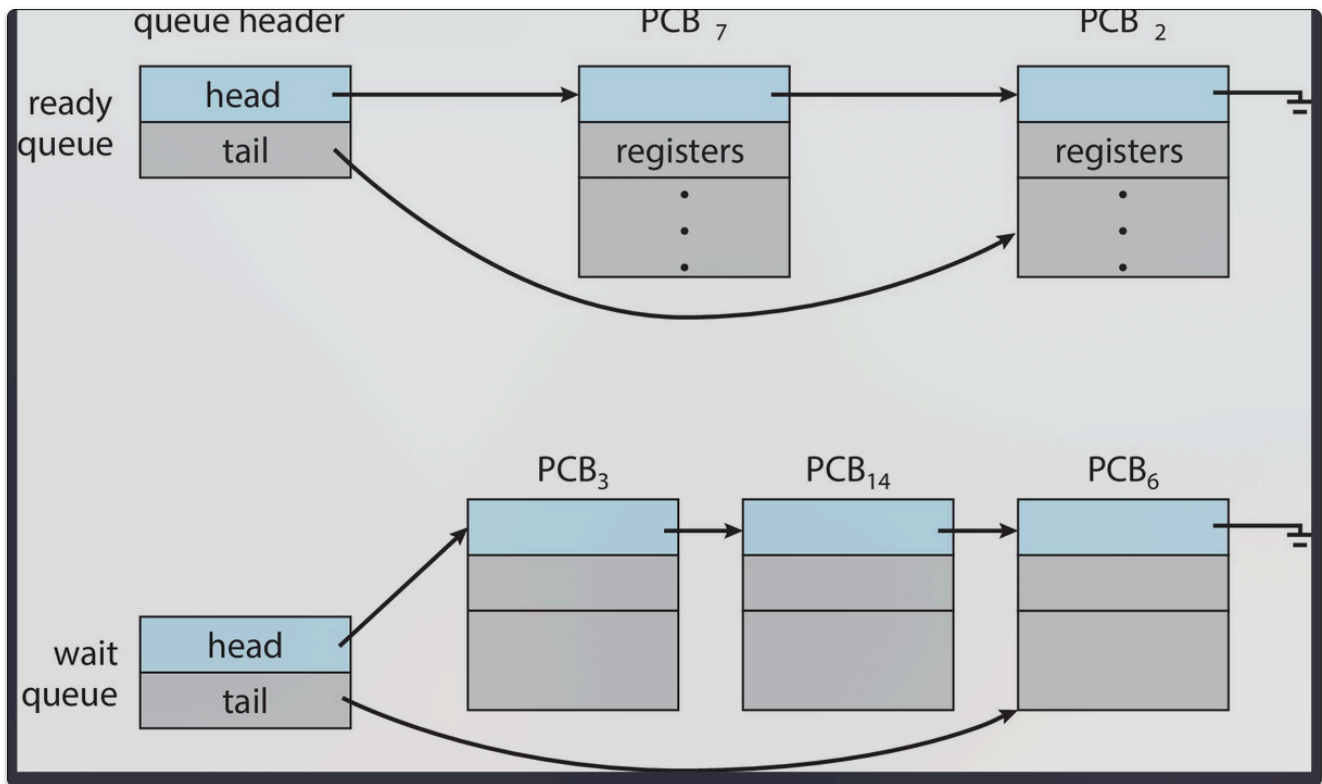


Image 5: Visualization of process scheduling

- Time slice expired
 - processes have limited time to execute

Context Switch

Context Switch

- CPU switches from one process to another

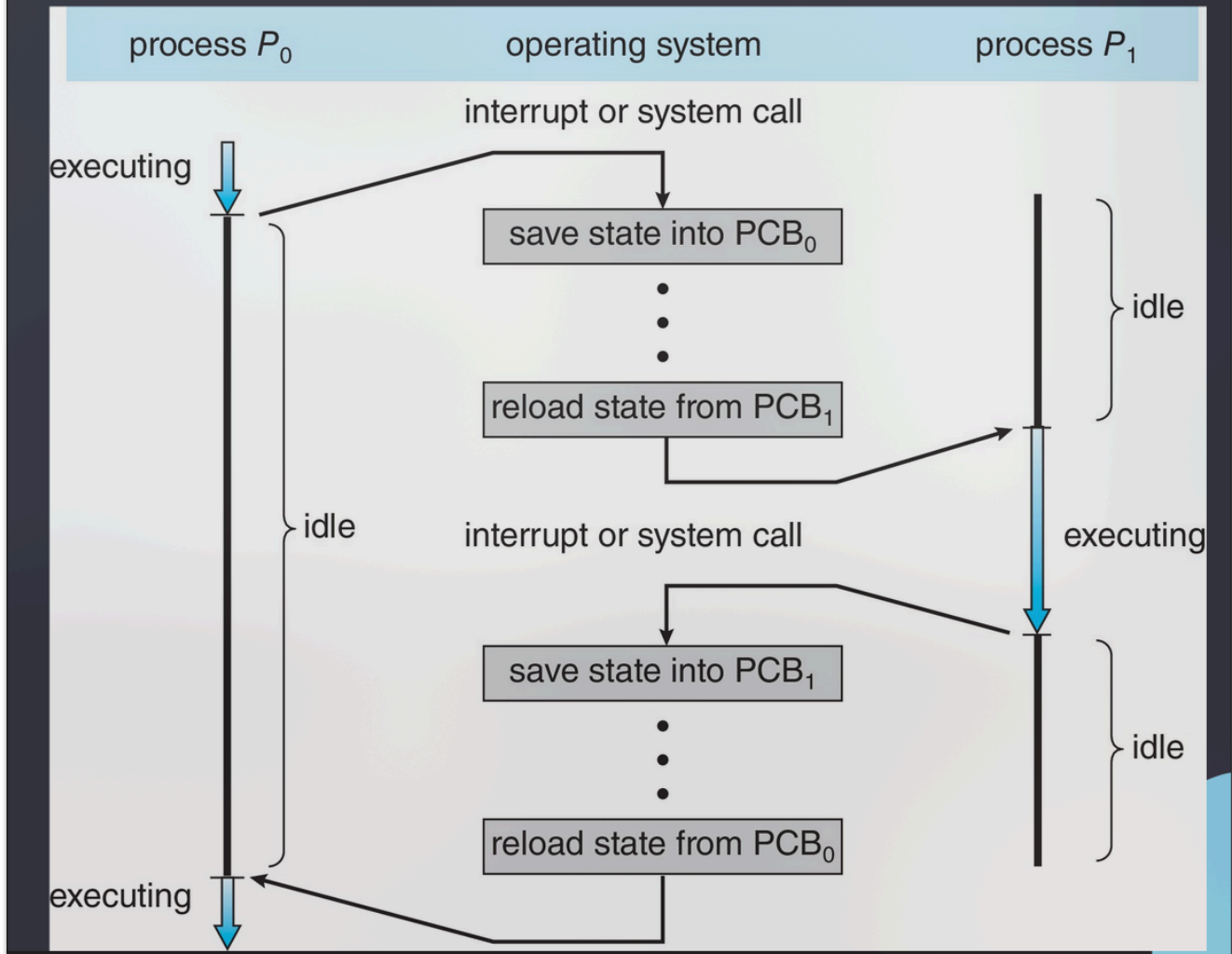


Image 6: Context switching visualization (take note of idle time, cause included in gantt chart)

- Imagine nag-aaral ka tapos biglang may nagchat sayo tapos chinat back mo.
 - If you switch from one process to the next, the system call of interrupting comes in, the operating system will save what it has done to process control block so you can easily resume before you got interrupted and context switched.
- Dependent on hardware - kung mabagal pc mo, mabagal context switching mo
- Context-switch time is pure overhead - system is not doing any actual work while switching
- Time dependent on hardware support

Multitasking in Mobile Systems

- Some mobile systems allow only one process to run, iba suspended
- Due to screen real estate - interface limits iOS provides for:

- single foreground process
- multiple background processes
- Android runs foreground and background with fewer limits
 - background process uses a service to perform tasks

The contrast with this in computer operating systems is that multiple process can be in the foreground process (i.e., you're playing cs2 and you have spotify playing or you're also search how to smoke mirage window)

Process Creation

- **Parent** process create **children process** which forms a tree of processes

The screenshot shows the bpytop interface with the process tree view selected. The tree is rooted at 'systemd (init)' and branches out into various user processes. Each process entry is accompanied by its PID, name, parent PID, and a list of child processes. The right side of the interface displays a table of process statistics.

Tree:	Threads:	User:	MemB	Cpu%
[--]-1 systemd (init)	1	root	8M	0.0
--22232 firefox	127	moonberry	933M	0.0
--22333 RDD Process (firefox)	5	moonberry	34M	0.0
--109375 file:/// Content (firefox)	28	moonberry	355M	0.0
--253440 Isolated Web Co (firefox)	29	moonberry	742M	0.0
--22407 WebExtensions (firefox)	27	moonberry	124M	0.0
--22451 Utility Process (firefox)	5	moonberry	29M	0.0
--22329 Privileged Cont (firefox)	27	moonberry	116M	0.0
--316003 Web Content (firefox)	18	moonberry	83M	0.0
--316055 Web Content (firefox)	18	moonberry	82M	0.0
--346255 Web Content (firefox)	18	moonberry	83M	0.0
--253910 Isolated Web Co (firefox)	27	moonberry	158M	0.0
--22309 Socket Process (firefox)	6	moonberry	26M	0.0
--22236 crashhelper	2	moonberry	3M	0.0
--357985 bash	1	moonberry	6M	0.2
--357990 bash	1	moonberry	4M	0.0
--357992 slurp	1	moonberry	30M	0.1
--354291 kitty	7	moonberry	114M	0.0
--354296 kitten	10	moonberry	22M	0.0
--354305 bash	1	moonberry	6M	0.0
--354406 python3	3	moonberry	48M	0.2
--734 waybar	23	moonberry	30M	0.0
--108035 Discord (discord)	39	moonberry	124M	0.0
--108169 Discord (exe)	46	moonberry	422M	0.1
--108097 Discord (exe)	11	moonberry	40M	0.0
--108221 Discord (exe)	6	moonberry	19M	0.0
--108038 Discord	1	moonberry	7M	0.0
--108091 Discord	22	moonberry	88M	0.0
--108039 Discord	1	moonberry	7M	0.0

Image 5: process tree in bpytop, I use arch btw

- Generally process identified and managed using process identifier (pid)
- Resource sharing options (depend on designer)
 - Parent and child share all resources
 - Children share subset of parent's resources
 - parent and child share no resources
- Execution options
 - parent and child execute concurrently
 - imagine adblocker in web browser

- parent waits until children terminate
- address space
 - child duplicates parent
 - child has program loaded into it
- in UNIX...
 - fork() system call create a new process
 - exec() system call after fork() to lead new program
 - Parent system calls wait() waiting for child to terminate

Process Termination

- **exit()** system call
 - return status data from child to parent (via wait())
 - Process' resources are de-allocated by operating system.
- Parent may terminate the execution of children process using **abort()** system call
 - child has exceeded allocated resources
 - task assigned to child is no longer required
 - the parent is exiting
- Some OS dont allow child to exist if parent is terminated
 - cascading termination - all children, grand children, are terminated
 - termination is initiated by OS
- The process may system call wait() for termination
 - pid = wait(&status)
- If no parent waiting (did not invoke wait()), process is a zombie
- If parent terminated without invoking wait(), the process is an orphan
 - inutusan ako pero di ako hinintay tapusin, di na ako connected sa process, therefore orphaned.
- Difference is that there exists a parent but is not waiting and no parent

Termination in Android - Importance Hierarchy

- most to least important:
 - foreground process
 - visible process
 - service process
 - background process
 - empty process
 - process that doesnt have anything
- Difference between foreground and visible process
 - imagine in the phone you have a game in the foreground. the taskbar when you pull down from the top is the visible process. not in the foreground but

can be seen

Interprocess Communication (IPC)

- Processes within a system may be independent or cooperating
- communicate with other processes by virtue of same message passing
- Cooperating process can affect or be affected by others
- Reasons for cooperating
 - information sharing
 - computation speedup
 - modularity
 - convenience
- Cooperating processes need IPC
- two models
 - shared memory
 - message passing